

Day 16: Constructors, Destructors & RAI

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Default & Parameterized Constructors

Constructors initialize an object when it's created. Without a constructor, member variables have **undefined values** (garbage). Always initialize:

```
class UartDriver {
public:
    UartDriver() : dev_(nullptr), initialized_(false) {} // Default: safe state
    explicit UartDriver(const struct device *dev) : dev_(dev), initialized_(false) {}
};
```

The **explicit** keyword prevents implicit conversion — always use it for single-argument constructors to avoid accidental object creation.

Destructors and Cleanup

The **destructor** (`~ClassName()`) runs automatically when an object goes out of scope or is deleted. Use it to release resources:

```
class GpioDriver {
    ~GpioDriver() {
        if (initialized_) {
            gpio_pin_configure(dev_, pin_, GPIO_DISCONNECTED); // Release pin
        }
    }
};
```

Copy vs Move Constructors

- **Copy constructor:** creates a new object as a copy of another. **Dangerous for hardware handles** — two objects would control the same peripheral.
- **Move constructor:** transfers ownership from a temporary to a new object.

For hardware driver classes, **delete both** to prevent accidental copying:

```
GpioDriver(const GpioDriver &) = delete; // No copy
GpioDriver &operator=(const GpioDriver &) = delete; // No copy-assign
```

RAII — The Most Important Embedded C++ Pattern

Resource Acquisition Is Initialization: acquire a resource in the constructor, release it in the destructor. This guarantees cleanup even if the function returns early or a sub-call fails:

```
class ScopedMutex {
public:
    explicit ScopedMutex(k_mutex &m) : m_(m) { k_mutex_lock(&m_, K_FOREVER); }
    ~ScopedMutex() { k_mutex_unlock(&m_); } // ALWAYS runs
    ScopedMutex(const ScopedMutex &) = delete;
private:
    k_mutex &m_;
};
// Usage:
{
    ScopedMutex lock(my_mutex); // Locked here
    // ... critical section ...
    if (error) return; // Lock still released! Destructor runs.
} // Unlocked here automatically
```

RAII for GPIO/DMA/Mutexes

Apply RAII to every resource that needs cleanup:

- **GPIO pin:** configure on construction, release (disconnect) on destruction
- **DMA channel:** allocate on construction, free on destruction
- **Mutex:** lock on construction, unlock on destruction (shown above)
- **Power domain:** enable on construction, disable on destruction

Deferred Initialization Pattern

Global objects are constructed before `main()` — but Zephyr drivers may not be ready yet. Solution: separate construction from initialization:

```
UartManager uart_manager;    // Constructed (safe, no hardware access)
// ...
uart_manager.init(uart_dev);  // Initialize when drivers are ready (inside main)
```

This avoids the **static initialization order fiasco** and aligns with Zephyr's initialization sequence.

Practice Tasks

1. Write a `ScopedIrqLock` RAII class that calls `irq_lock()` on construction and `irq_unlock()` on destruction
 2. Create a `GpioPinGuard` that configures a GPIO as output on construction and disconnects it on destruction
 3. Try to copy a driver class with `= delete` copy constructor — observe the compile error
 4. Use `= default` and `= delete` to control which special member functions are generated
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day16
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day16/main.cpp](#)